PYTHON

VARIABLES, KEYWORDS AND OPERATORS

2.1 VARIABLES

In python, variables are used to store values that can be used later in a program. It acts as a container for storing the values. They are also known as **Identifiers.**

Eg. name = “Gracy”

Here name = variable name, “Gracy” = value.

We can also update or re-assign the value of the variable by,

Age = 25

Age = 26 #updating the value of age variable.

2.2 IDENTIFIERS IN PYTHON

In python, an identifier is a name used to identify a variable function, class module, or any other user-defined object. An identifier can be made up of both uppercase and lowercase alphabets, digits and underscores. However it must start with a letter or an underscore.

Rules for Identifiers:

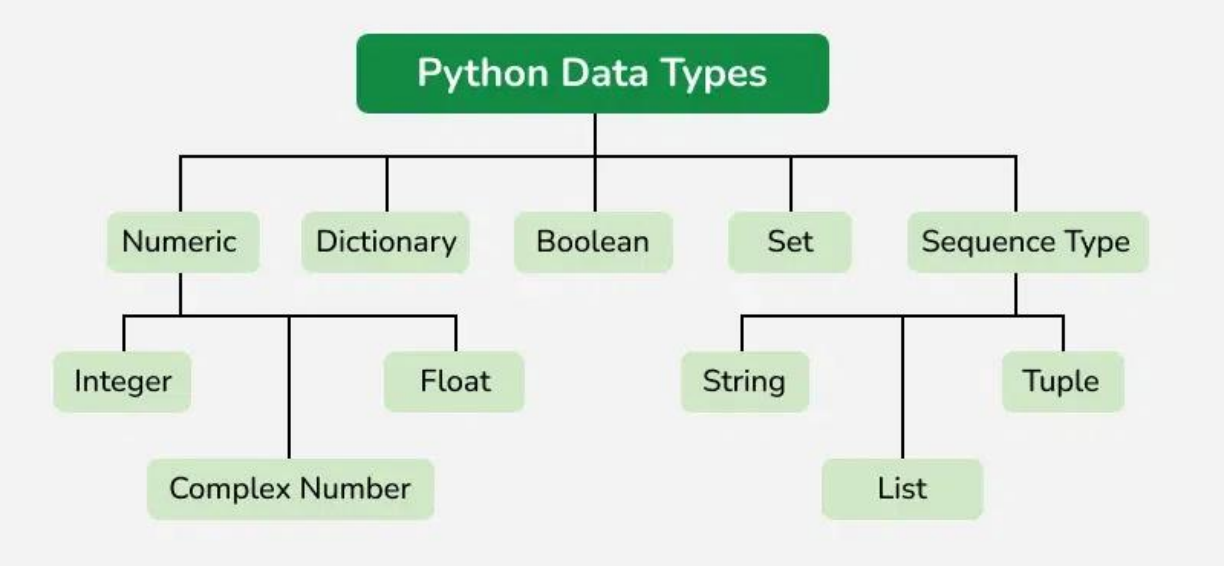
- Identifiers cannot contain spaces, and not start with digits or any special characters except underscore.
- Python is case-sensitive language, so identifiers also follows case sensitivity.
- Python has reserved words, also known as keywords, that have predefined meanings in the language. These words cannot be used as identifiers. Eg. If, else, while etc.
- Should start with uppercase or lowercase alphabets or underscore (_).

Eg. my_variable , count, total_sum, age12, Myclass

2.3 DATATYPES IN PYTHON

Datatypes specifies what type of data that we are going to store on the variable, They determine the nature of the data and the operations that can be performed on them.

Python is dynamically typed, meaning there is no need to declare type explicitly. Type is decided at runtime.



1. **NUMERIC:**

Python supports different numerical data types, including integers(whole numbers), floating-point numbers(decimal numbers) and complex numbers (numbers with real and imaginary parts)

i. **Integers (int) :** Integers represent whole numbers without any fractional part. For example, age = 25.
ii. **Floating-point numbers (float) :** Floating-point numbers represent numbers with decimal point or fractions. For example, pi = 3.14.
iii. **Complex numbers (complex) :** Complex numbers have a real and imaginary part. For example, z = 2+3j where j is the suffix with it.

2. DICTIONARY:

Dictionaries are key-value pairs enclosed by curly braces. Each value is associated with a unique key, allowing for efficient lookup and retrieval. For example : person = {‘name’ : ‘John’, ‘age’ : 25, ‘city’ : ‘New york’}

3. **BOOLEAN:**

Boolean (bool) : Booleans represents true or false values. They are used for logical operators and conditions. For example, is_valid = True.

4. **SET:**

Sets (set) : Sets are unordered collections of unique elements enclosed in curly braces. They are useful for mathematical operations such as union, intersection, and difference. For example, Fruits = {‘apple’, ‘banana’,’orange’}

5. **SEQUENCE TYPE:**

Sequence represent a collection of elements and include data types like strings, lists and tuples. Strings are used to store textual data, while lists and tuples are used to store ordered collections of items.

i. Strings (str): Strings represent sequences of characters enclosed within single or double quotes. For example, name = ‘john’
ii. Lists (list): Lists are ordered sequences of element enclosed in square brackets. For each element be of any data type. For example, X = [100, ‘a’, ‘apple’, 32.3]
iii. Tuples (tuple): They are similar to lists but are immutable, meaning their elements cannot be changed once defined. They are enclosed in () parenthesis.
x = (12, 45, 52, 63, 200)

We can check the type by using the **type()** method. Type() method returns the type of the variable.

Eg.

x = 50 → type(x) # int

x = 60.5 → type(x) # float

x = "Hello World" → type(x) # string

x = ["geeks", "for", "geeks"] → type(x) # list

x = ("geeks", "for", "geeks") → type(x) # tuple

2.4 KEYWORDS

- Keywords in python are special words that are having specific meanings and purposes with the python language. They are reserved and cannot be used as variable names or identifiers.

List of all keywords in python: (35)

and	***continue***	***finally***	***is***	***raise***
as	***def***	***for***	***lambda***	***return***
assert	***del***	***from***	***nonlocal***	***True***
async	***elif***	***global***	***None***	***try***
await	***else***	***if***	***not***	***while***
break	***except***	***import***	***or***	***with***
class	***False***	***in***	***pass***	***yield***

2. 5 OPERATORS IN PYTHON:

Operators in python are symbols or special characters that are used to perform specific operations on variables and values. Python provides various types of operators to manipulate and work with different data types. Here are some important categories of operators in python.

i. Arithmetic Operators [+, -, *, /, %, **, //]
ii. Assignment Operators [=, +=, -=, *=, /=, %=, //=. **=, &=, |= …]
iii. Comparison Operators [==, !=, >, <, >=, <=]
iv. Logical Operators [and, or, not]
v. Bitwise Operators [&, |, ^, ~, <<, >>]
vi. Membership Operators [in, not in]
vii. Identity Operators [is, is not]

Link : https://www.w3schools.com/python/python_operators_arithmetic.asp

2.6 TYPE CONVERSION

Type conversion means changing one data type into another, python supports two types,

- Implicit Type conversion (automatic)
- Explicit Type conversion (Manual / Type casting)

Implicit Type Conversion:

- Done automatically by Python interpreter
- Occurs when different data types are used in an expression
- Python converts smaller data type to larger data type to avoid data loss

```
x = 10      # int
y = 2.5     # float

result = x + y
print(result)        # 12.5
print(type(result))  # float
```

- Int is converted to float
- Final result becomes float

Explicit Type Conversion (type casting):

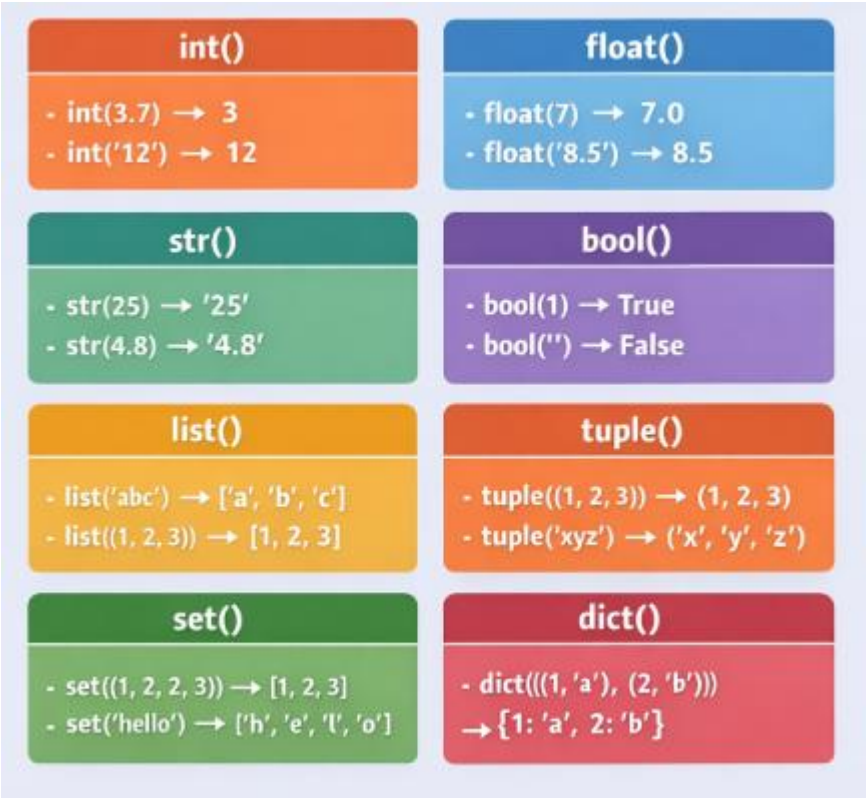
- Done manually by the programmer
- Uses built-in functions

eg.

```
x = int(1)   # x will be 1
y = int(2.8) # y will be 2
z = int("3") # z will be 3
```

```
x = float(1)     # x will be 1.0
y = float(2.8)   # y will be 2.8
z = float("3")   # z will be 3.0
w = float("4.2") # w will be 4.2
```

```
x = str("s1") # x will be 's1'
y = str(2)    # y will be '2'
z = str(3.0)  # z will be '3.0'
```



2. 7 PRINTING STATEMENTS IN PYTHON

Link: https://www.geeksforgeeks.org/python/python-output-using-print-function/

Syntax : print(value(s), sep= ' ', end = '\n', file=file, flush=flush)

Parameters:

- **value(s):** Any value, and as many as you like. Will be converted to a string before printed
- **sep='separator' :** (Optional) Specify how to separate the objects, if there is more than one.Default :' '
- **end='end':** (Optional) Specify what to print at the end.Default : '\n'
- **file :** (Optional) An object with a write method. Default :sys.stdout
- **flush :** (Optional) A Boolean, specifying if the output is flushed (True) or buffered (False). Default: False

Return Type: It returns output to the screen.

2. 8 PRACTICE PROBLEMS:

1. https://www.geeksforgeeks.org/problems/start-coding-python--093158/1?selectedLang=python3

2. https://www.geeksforgeeks.org/problems/learn-to-comment-python/1?selectedLang=python3

3. https://www.geeksforgeeks.org/problems/print-in-python/1?selectedLang=python3

4. https://www.geeksforgeeks.org/problems/multi-printing--153452/1?selectedLang=python3

5. https://www.geeksforgeeks.org/problems/int-str--164410/1?selectedLang=python3

6. https://www.geeksforgeeks.org/problems/input-in-python/1?selectedLang=python3

7. https://www.geeksforgeeks.org/problems/logical-operators-1597313687--171003/1?&selectedLang=python3

8. https://www.geeksforgeeks.org/problems/bitwise-operators-1597314674--135255/1?&selectedLang=python3

9. https://www.geeksforgeeks.org/problems/the-modulo-task1736/1&selectedLang=python3